Aggregate Indoor Localisation

Giorgio Audrito[1[0000−0002−2319−0375]], Leonardo Bertolino[1], Ferruccio Damiani[1[0000−0001−8109−1706]], and Gianluca Torta[1[0000−0002−4276−7213]]

Department of Computer Science, University of Turin, Torino, Italy
{giorgio.audrito,ferruccio.damiani,gianluca.torta}@unito.it
leonardo.bertolino@edu.unito.it

Abstract. Accurate indoor localisation is a key enabling technology for many distributed systems, including collective robotics, smart environments, and user wearable devices. In these settings, localisation must often be achieved without central coordination, under limited sensing capabilities, and in the presence of dynamic environmental changes. Collective indoor localisation algorithms address these challenges by exploiting local interactions among devices to collectively infer positions. However, many existing approaches implicitly assume relatively stable conditions and offer limited guarantees on adaptability and resilience.
In this paper, we investigate cooperative indoor localisation from a self-adaptive perspective. We reformulate two established localisation algorithms within the aggregate programming framework, and propose novel variants of them to improve their performance. We evaluate our proposals through extensive simulation, analysing error and communication cost across a wide range of scenarios, including recovery from disruptive events. Our results uncover the trade-offs between accuracy, resiliency and efficiency, shedding light on the design space of what we call aggregate indoor localisation algorithms.

Keywords: Aggregate programming · Cooperative indoor localisation · Self-stabilisation.

1 Introduction

Localization is a fundamental enabling functionality for numerous distributed systems operating in indoor environments, including collective robotics, sensor networks, cyber-physical systems, and smart environments. In such scenarios, knowledge of spatial position enables coordination, motion planning, and efficient resource management. Unlike outdoor scenarios, where satellite systems such as GPS provide a consolidated global infrastructure for positioning, indoor environments render such a solution impractical or highly unreliable [9]. Satellite signals are significantly attenuated by building structures, are subject to reflections and interference that degrade accuracy, and, in many cases, are not received at all in the absence of direct visibility of the satellites. Moreover, even when the signal is available, the accuracy provided is generally not sufficient for applications requiring metric or sub-metric precision in complex enclosed spaces.

Consequently, indoor localization cannot rely on a pre-existing global infrastructure, but must instead be achieved through distributed mechanisms based on local information. This requirement is particularly relevant in systems composed of autonomous and potentially mobile devices, which communicate predominantly with physically nearby nodes and operate asynchronously [13]. In such contexts, position must be estimated through cooperative processes in which spatial information is progressively propagated across the network through multi-hop interactions. Despite the extensive literature on cooperative localization, many existing algorithms assume relatively stable conditions, such as reliable connectivity, largely static network topologies, and the absence of significant perturbations. In real scenarios, however, distributed indoor systems are inherently dynamic: devices may join or leave the network, experience temporary failures or variations in measurement quality, and the communication topology may evolve over time. Under these conditions, traditional algorithms can be slow to converge, unstable, or unable to automatically adapt to changes.

The problem we address, therefore, is how to design cooperative localization algorithms that are not only accurate, but also resilient and capable of self-adapting to network dynamics and environmental perturbations without requiring external intervention. In particular, we are interested in solutions that are fit for decentralised, asynchronous systems with proximity-based interactions. We revisit two well-known algorithms from the literature: *distance vector–based positioning* and *non-Bayesian cooperative localisation* [17,9], expressing them within the *aggregate programming* [6] framework, which is designed to support decentralised coordination in mutable environments. This translation enables the algorithms to naturally react to changes and *self-stabilise* towards consistent results without external intervention. We then introduce improved variants of the two algorithms above, working both with and without inter-node distance estimates, and aimed at improving localisation accuracy and communication efficiency.

We evaluate the proposed algorithms through an extensive selection of simulations in dynamic multi-hop scenarios, varying the communication radius, device speed, and measurement noise level. The analysis considers both the localization accuracy and the computational and communication cost, in a scenario which comprises nodes failing and joining to assess resilience. The results highlight an improved trade-off between precision, capacity for self-adaptation, and efficiency of the proposed variants with respect to the original versions of the algorithms. Overall, the evaluation provides a comprehensive guide for choosing the best algorithm for any given scenario at hand.

This paper makes the following contributions:

- **Formalisation in aggregate programming.** We reformulate two established indoor localisation algorithms (distance vector–based positioning and non-Bayesian cooperative localisation) within the aggregate programming paradigm, providing an elegant and compositional specification in the eXchange Calculus (XC) [2] for both of them.

- **Stabilisation guarantees.** We prove that the aggregate reformulation induces (self-)stabilising behaviour, endowing the algorithms with formal convergence guarantees that were not directly available in their original decentralised formulations, particularly for distance vector–based approaches.
- **Algorithmic variants and trade-off analysis.** We design several variants, operating both with and without inter-node distance estimates, and systematically evaluate their behaviour under noise and network disruptions, including recovery from transient perturbations. We provide a comparative study of localisation error and communication cost, highlighting the trade-offs among accuracy, message size, computational cost, and recovery performance in dynamic multi-hop settings.

The remainder of the paper is structured as follows. Section 2 states the problem of cooperative indoor localisation in decentralized multi-hop networks, introducing the system model, the role of anchor nodes, and the self-stabilisation requirement adopted. Section 3 presents the necessary background on the Aggregate Programming paradigm and on the main multi-hop geometric algorithms for indoor localisation. Section 4 describes our aggregate reformulation of the two selected algorithms, illustrating their operation with pseudocode and introducing the proposed variants to improve convergence and robustness. Section 5 presents the experimental evaluation, analysing the results in terms of accuracy, communication cost, and adaptability in dynamic scenarios. Section 6 concludes with final remarks and directions for future work.

2 Problem Statement

We consider the problem of cooperative indoor localisation in decentralised distributed systems composed of multiple devices interacting through proximity-based communication. The system is modelled as a multi-hop network, where each node can exchange information only with neighbours within communication range, and no central coordination entity is assumed. A subset of nodes, called *anchors*, have a fixed and known position in a common reference frame. All other nodes aim to estimate their own position by exploiting local interactions with neighbours. Depending on sensing capabilities, nodes may or may not have access to (noisy) estimates of the distance to their neighbours. The network and the environment are assumed to be mutable: nodes (including anchors) may move, join or leave the network, and communication may fail.

A first, obvious requirement for any proposed solution is that it is *stabilising*, i.e., that after a finite amount of time every node in the network reaches a stable value. However, to address the mutability of the network, localisation algorithms should ideally not only provide accurate position estimates under stable conditions, but also be able to adapt to changes as the ones previously mentioned without requiring explicit reinitialisation or external control. It is then highly desirable that the solutions are also *self-stabilising*, a stronger property than mere stabilisation. Informally, a self-stabilising algorithm is one that, starting from an arbitrary internal state (that may be influenced by e.g. a recent

input change), provided that its input data and environment stay constant for enough time, is guaranteed to converge to the correct outcome for its input and environment, independently of the algorithm state before stabilisation.

3 Related Work and Background

3.1 Aggregate Programming

The Internet of Things (IoT) is leading to an exponential growth in the number of connected devices, such as smartphones, drones, sensors, and embedded systems, which primarily rely on proximity-based local communications. Programming approaches centered on individual devices are becoming burdensome for developers, who are forced to directly manage communication, coordination, and behavioral composition, resulting in issues of modularity and reusability.

To address these challenges, Aggregate Programming (AP) emerged as a paradigm for programming the collective behavior of sets of distributed devices. Such devices operate autonomously and asynchronously, do not share memory, and typically communicate only with physically neighboring nodes. The entire computational environment is viewed as a single entity—more precisely, a region—within which the technical details of individual devices are abstracted away. Rather than reasoning about individual devices, programmers reason about the space those devices occupy, using data constructs that span across space and time in the region [6].

Despite this global abstraction, the program is executed locally by the individual devices, which implicitly coordinate to ensure robustness and resilience, even in the presence of failures or dynamic changes in the system. Local execution on each device follows a cyclical round-based schedule, also known as *sense–compute–send–sleep*. In each cycle, the device initially acquires information from local sensors and messages received from neighboring nodes in the previous cycle (*sense*). Based on this data, local computation is then performed (*compute*), at the end of which messages may be produced for adjacent devices (*send*). Once execution is complete, the device remains inactive until the start of the next cycle (*sleep*).

For AP, the reference languages are the Field Calculus (FC) [16] and its recent evolution, XC [2]. The latter is essentially a typed lambda calculus with a fundamental `exchange` operator to provide implicit communication between nearby devices. Within XC, communication between nearby devices does not occur via direct messages but through a special kind of data called *nvalue* (for neighboring value). The nvalue is a map that associates device identifiers with literal values and has a *default* value that is assigned to all other devices, including those that are not known as neighbours by the sender. Neighbouring values are used to represent both values received from directly connected devices, and to represent values that need to be sent back to them. The `exchange` primitive has the following form:

$$\texttt{exchange}(\texttt{e}_i\texttt{, (n) => return e}_r\texttt{ send e}_s\texttt{))}$$

A device evaluates the expression $\mathtt{e}_i$ to obtain an initial literal value ℓ_i. Then, it substitutes the lambda argument `n` with an nvalue that has ℓ_i as *default*, and associates neighbours to the messages received from them for this exchange expression. The expression returns a value $\mathtt{e}_r$ to be used on the device, and simultaneously generates the nvalue $\mathtt{e}_s$ to be sent to neighbors, which will use it in their next round to produce their argument `n`. In the pseudo-code in the following sections, we will use some convenient specializations of `exchange`:

```
def share(init, update) {
  exchange(init, (n) => val v = update(n); return v send v)
}

def nbr(init, value) {
  exchange(init, (n) => return n send value)
}

def rep(init, update) {
  exchange(init, (n) => val v = update(self(n)); return v send v)
}
```

The `share` specialization derives a single local value (i.e., an nvalue with only the default and no special values for neighbours) by applying an `update` function to `n`, then returns it and sends it to all the neighbours. The `nbr` specialization returns the received nvalue, and sends a local value received as parameter to all the neighbours. Finally, the `rep` specialization returns and sends to itself an `update` of the value `self(n)` for the current device in `n` (i.e., it propagates local status between rounds).

In a program, there may be multiple `exchange` calls in the same round, and it is necessary that each such call receives the correct values from neighbours. A mechanism called *alignment* is used to ensure that each `exchange` receives the correct neighbouring values corresponding to the same `exchange` expression in neighbours' rounds (both as position in source code syntax tree, and as position in the current stack trace of function calls). This ensures that there are no conflicts between multiple exchanges in the same program. This important feature enables *compositionality*, i.e., an XC program can reuse (one or several) functions that contain `exchange`, that will just work as expected.

In addition to the fundamental exchange construct and its specializations (`share`, `nbr`, `rep`), XC also provides a higher-level mechanism for managing dynamic spatial computations, called `spawn` [3,7], which forms the basis of the *aggregate processes* model. The function `spawn`$(\mathtt{e}_b, \mathtt{e}_k, \mathtt{e}_i)$ allows the initiation and management of sets of distributed processes indexed by keys. The expression $\mathtt{e}_b$ models the process behaviour and is a function that, given an identifying key and an input, returns a pair composed of an output value and a Boolean indicating whether the process should propagate to neighbours or terminate in the current node. The expression $\mathtt{e}_k$ defines the set of process keys for which new processes should be spawned on the current device, while $\mathtt{e}_i$ represents an additional input provided to the process behaviour. The result of spawn is a map

that associates each key of an executed process with its corresponding output value.

From a semantic point of view, depending on whether each device decides to keep a process instance active, that instance is automatically propagated to neighbouring devices or not, giving rise to a distributed computation that extends through space and evolves dynamically over time. Furthermore, processes influence the alignment mechanism: an exchange operation executed within a process can only be aligned with corresponding operations belonging to the same process (with the same key) on neighbouring devices. This ensures isolation between parallel computations and allows multiple independent processes to execute simultaneously, preserving modularity and composability.

3.2 Multi-Hop Indoor Localisation

The localization of devices (hereafter also referred to as *nodes*) in indoor environments represents a significant challenge, as knowledge of their positions plays a central role in many network applications and functionalities, such as the need to associate collected measurements with a spatial context. The use of GPS is not applicable indoors, due to the attenuation (or complete blocking) of the signal caused by walls and roofs. Even in outdoor scenarios, however, GPS is not always a suitable solution, because, e.g., of costs, power consumption, inaccuracy [9], or simply in GPS-denied areas. Thus, most of the techniques considered here can also be relevant to outdoor settings.

To allow localization without relying on GPS (or similar systems), algorithms based on geometric methods have been introduced, enabling the estimation of node positions relying on other information. Among the most widely used geometric methods are trilateration, and its variant named multilateration. Trilateration [14] is a geometric technique that allows the position of a node to be determined by knowing its distances from three reference nodes (anchors) with known positions. The node position corresponds to the intersection of three spheres, each centered at a reference node and with a radius equal to the measured distance from the unknown node. The determination of the distance between two nodes is itself an interesting problem that is typically addressed with wireless technologies, such as BLE, WiFi, or UWB. Among these technologies, UWB has been reported to achieve positioning accuracy with an error bound of approximately $10cm$ [10]. Explicitly considering the techniques for distance estimation is out of the scope of the present paper. However, we will introduce controlled random estimation errors in the experiments to make them more realistic.

Position estimation through multilateration [1] is formulated as an optimization problem, in which the position of the unknown node is determined by minimizing the Root Mean Square Error (RMSE) between the distances measured from *anchor* nodes and the distances corresponding to the estimated position. This approach makes it possible to achieve robust localization even in the presence of noisy distance measurements, by combining multiple measurements and, if necessary, assigning them appropriate weights. The numerical solution of this least-squares problem can be obtained using the Levenberg–Marquardt method

[5]. When at least two anchors are present, the function enters an iterative loop: at each iteration, it computes the total position error by comparing the estimated distances with the actual ones. It then determines the direction in which to move, taking into account both the gradient and the direction in which the error increases. Based on this information (and the *step size*), a new position estimate is computed, for which the error is evaluated again. If the error increases, the new estimate is rejected and the step size is reduced; conversely, if the error decreases, the estimate is accepted and the step size is increased. This adaptation mechanism guarantees the stability and convergence of the algorithm toward a solution that minimizes the error.

Several studies have addressed *multi-hop* localization, that is, localization in networks where nodes can share information with their neighbours to support the localization task where the availability of anchor nodes is limited. Among these, [11] proposes a combined multi-hop localization and time synchronization approach (the latter is necessary for correctly estimating the distances between nodes) using a single anchor node, highlighting the challenges and strategies for localization under resource-limited conditions. Yet, when three or more anchor nodes are available, it is possible to exploit the geometric methods discussed previously. A classic algorithm known as *DV-Hop* [9] (where DV stands for Distance Vector) is structured into three main phases. First, nodes exchange information with their neighbors to determine the distance in hops from the anchors. Second, the anchors, knowing their mutual Euclidean distances, calculate a correction factor to estimate the average hop length. Finally, nodes use this information to perform multilateration and estimate their own position. Note that DV-hop is considered to be a *sequential* algorithm [18], due to unidirectional share of information in the first two phases, followed by local position estimation in the third phase. Another classic approach is *NB-Coop* (which stands for Non-Bayesian Cooperative) [17], in which nodes iteratively update their own position by comparing the estimated distances from neighbors with those calculated from their current estimated positions. It is similar to the multilateration method described above, but with notable differences that will be explained later. One key difference is that a node's neighbors are generally not all anchors. The approach is defined as non-Bayesian because it treats node positions as unknown input parameters, rather than random variables with given probability distributions.

Finally, there is a vast literature on more complex multi-hop geometric algorithms, ranging from Bayesian cooperative message passing [17] to distributed convex optimization [12]. In this paper we will focus on the AP implementation and optimization of the DV-hop and NB-Coop classic algorithms mentioned above. We remark that, to the best of our knowledge, no investigation has been performed in literature on the usage of these algorithms in mutable environments, and in particular on self-stabilisation. Instead, works in literature assume a single execution until convergence on a fixed topology. The convergence proof for the classic NB-Coop algorithm to (local or) global minima can be re-interpreted as a (self-)stabilisation proof for the algorithm. However, classic

Non-Bayesian Cooperative Localisation

Input: $initPos, isAnchor, nbrDist$

1: **share starting from** $pos \leftarrow initPos$ **update**
2: $anchorInfo \leftarrow \{(pos_i, nbrDist_i)$ for each neighbour $i\}$
3: **if** $isAnchor$ **then**
4: **return** $pos \leftarrow$ known position
5: **else**
6: **return** $pos \leftarrow gradientDescent$ given $anchorInfo$ and pos
7: **end if**
8: **end share**

```
def nbcoop(initPos, isAnchor, nbrDist) {
  share(initPos, (pos) =>
    val anchorInfo = nfold(cons, tuple(pos, nbrDist), nil);
    if (isAnchor) { knownPos } else {
      gradientDescent(anchorInfo, self(pos))
    }
  )
}
```

Fig. 1. Non-bayesian cooperative localisation algorithm (above) and corresponding XC code (below).

formulations of DV-hop algorithm do not self-stabilise, since these do not include a mechanism to remove knowledge about anchors that exited the network.

4 Aggregate Indoor Localisation Algorithms

4.1 Non-Bayesian Cooperative Localisation

The Non-Bayesian cooperative localisation algorithm [17] proceeds by iterative refinement of a working estimated position. We consider a uniformly random position in the monitored area as the initial estimate *initPos*. As the iterations proceed, the estimate is progressively refined thanks to the exchange of information with neighbours. This strategy of progressive refinement can be naturally expressed as an aggregate algorithm in XC, given in Figure 1. For presentation purposes, we also include a pseudo-code representation of the algorithm, which we discuss in the following, leaving the interpretation of the XC code to the interested reader.

At each round, each node receives from its neighbours their estimated position *pos* (through the **share** function[1] in line 1) and the measured distance *nbrDist* (algorithm input) that separates the current device from them, both as

[1] While in the pseudo-code **share** has a slightly different syntax than as presented in Section 3.1, its *init* value and *update* can be easily recognized. A similar observation applies to **rep** and **spawn** in pseudo-code given later.

neighbouring values. If the node is an anchor (line 3), no calculation is needed since its position is already known (line 4). For non-anchor nodes, this information is processed by the *gradientDescent* function (line 6), which is the technical heart of the algorithm. This routine compares measured distances $nbrDist_i$ with the geometric distances of position estimates pos_i, and updates the current position estimate *pos* accordingly, moving it closer to or farther from the neighbours in linear proportion to the discrepancies detected.[2] This procedure corresponds to a gradient descent of the linearised least-squares error (hence the name of the function), which is an approximation of the actual least-squares error that is simpler to optimise.

A key role in this algorithm is played by the anchor nodes, which provide the absolute spatial references necessary to constrain the localization process. In the absence of anchors, the algorithm still converges, but only to a relative solution, which may differ from reality by any global translation or rotation. At least three anchors are needed to remove all ambiguity, and an adequate distribution of more anchors helps to speed up the iterative process and reduce errors.

In order to improve convergence speed and accuracy, we also consider a variant of the algorithm which uses a *multilateration* function instead of *gradientDescent* to update the position of the nodes, substituting line 6 as follows:

6: **return** *pos* ← *multilateration* given *anchorInfo* and *pos*

This variant differs in the strategy adopted for updating the position of non-anchor nodes. In the standard version based on *gradientDescent*, the node's position is updated incrementally, following the direction reducing the linearised least-squares error between the estimated and measured distances. By contrast, in this *multilateration*-based variant, we directly compute the position minimising the actual least-squares error with the Levenberg–Marquardt method. This approach requires more computation per each round, but may allow for more precise estimates (as we do not resort to the linearised approximation). Further, since we directly compute the optimal position, this method can allow for faster convergence time. However, in general faster convergence may also increase instability in case of noisy or fast-changing input, so the practical effectiveness of the approach has to be evaluated through simulation. We remark that in both variants, the update is iterative and local: at each step, the node modifies its position based on the current state and the information received from neighbours, progressively integrating the contributions of all available measurements. Furthermore, both variants can be shown to be stabilising.

Theorem 1. *Algorithm 1 is stabilising in both the* gradient descent *and* multilateration *variants.*

Proof. Define the total error of the network as the sum of the square of the differences between all measured distances $nbrDist_i$ and the corresponding dis-

[2] This can be physically interpreted as the overall nudge that would result in a movable point at the estimated position *pos*, that is connected with elastic springs to fixed points at the estimated positions of neighbours pos_i, where the length at rest of the springs is given by the measured distances $nbrDist_i$.

Distance Vector-Based Positioning

Input: $initPos, isAnchor, nbrDist$

rep starting from $pos \leftarrow initPos$ and $corr \leftarrow 1$ **update**
  $\{dist_a, pos_a, corr_a$ for each close anchor $a\} \leftarrow anchorsDPC$ algorithm
  if $isAnchor$ **then**
    $totApxDist \leftarrow$ sum of $dist_a$ if $dist_a$ finite
    $totTrueDist \leftarrow$ sum of $\|pos - pos_a\|$ if $dist_a$ finite
    return $pos \leftarrow$ known position, $corr \leftarrow totTrueDist/totApxDist$
  else
    $anchorInfo \leftarrow \{(pos_a, dist_a \cdot corr_a)$ for each close anchor $a\}$
    return $pos \leftarrow multilateration$ given $anchorInfo$ and pos, $corr \leftarrow 1$
  end if
end rep

```
def dv(initPos, isAnchor, nbrDist) {
  rep(tuple(initPos, 1), (old) =>
    val [pos,corr] = old;
    val [D,P,C] = anchorsDPC(pos, corr, isAnchor, nbrDist, param);
    if (isAnchor) {
      val totApxDist = nfold(+, mux(D < ∞, D, 0));
      val totTrueDist = nfold(+, mux(D < ∞, norm(pos - P), 0);
      tuple(knownPos, totTrueDist / totApxDist)
    } else {
      val anchorInfo = tuple(P, D*C);
      tuple(multilateration(anchorInfo, pos), 1)
} ) }
\midskip
```

Fig. 2. Distance Vector-based positioning algorithm (above) and corresponding XC code.

tances of the estimated positions $\|pos_i - pos\|$. During each round, the algorithm can only (weakly) decrease the error of the current device, thus reducing also the overall error of the whole network. Since the total error is a positive number represented in finite precision, it cannot decrease forever, and a minimum (possibly larger than 0) will be eventually reached. Once that happens, the algorithm will have stabilised.

However, neither of the variants is necessarily *self-stabilising*, that is, so that the stabilisation result does not depend on the initial conditions. In fact, the literature shows [8] that the optimisation may get stuck on local minima, never reaching the global minima, depending on the initial conditions. Special variations have been developed to ensure a single minimum [8], often at the expense of convergence time. In our simulations, however, we never encountered such local minima, proving experimentally that they are unlikely for randomised graphs. Thus, we decided not to consider those variations in this paper.

R-Close Anchors DPC

Input: $pos, corr, isAnchor, nbrDist, R$

1: **spawn for each** anchor a **doing**
2: $dist_a \leftarrow distance$ from a using $nbrDist$
3: $pos_a, corr_a \leftarrow broadcast$ of $pos, corr$ from a using $dist_a$
4: **exit process if** $dist_a \geq R$ **else**
5: **return** $dist_a, pos_a, corr_a$
6: **end spawn**
7: **return** $\{dist_a, pos_a, corr_a$ for each close anchor $a\}$

```
def anchorsDPC(pos, corr, isAnchor, nbrDist, R) {
  spawn((a) => {
    val dist = distance(uid() == a, nbrDist);
    val [pos,corr] = broadcast(dist, tuple(pos,corr));
    tuple(tuple(dist,pos,corr), dist < R)
  }, if (isAnchor) {cons(nil,uid())} else {nil})
}
```

Fig. 3. R-close anchors DPC algorithm (above) and the corresponding XC code (below).

4.2 Distance Vector-Based Positioning

In the DV-Hop algorithm [9], each node progressively builds an estimate of its own position based on information received from anchor nodes. In particular, each node maintains a list of known anchors and their respective estimated distances, which are iteratively updated through the propagation of information across the network. This process of progressive dissemination and update can be naturally expressed as an aggregate algorithm in XC (see Figure 2), as shown also in the simplified pseudo-code.

At each round, every node internally maintains a copy of the state values *pos* (position) and *corr* (correction factor), through the **rep** function in line 1. Here, *pos* represents the node's current estimate of its spatial position, and *corr* is a scaling factor computed by anchor nodes to rescale estimated distances to more closely match Euclidean distances. The initial values are *pos* = *initPos* (which may be a fixed or random position in the deployment area) and *corr* = *1* (no rescaling). At each round, these values are recomputed from the information received from anchors and stored as persistent state for the next round, keeping track of current estimates during the algorithm execution.

At line 2, we collect *DPC* information (where DPC stands for estimated relative Distance, absolute euclidean Position, Correction factor) from anchors. We accomplish this according to two main variants. In the first, closer to [9], which we call *R-Close Anchors* (Figure 3), each node considers all anchors whose estimated distance does not exceed a predefined threshold R, stopping the propagation of information beyond this range. In the second variant, called *K-Closest Anchors* (Figure 4), each node maintains an updated list of reachable anchors,

K-Closest Anchors DPC

Input: $pos, corr, isAnchor, nbrDist, K$

1: **share starting from** $dpc \leftarrow \{UID \mapsto (0, pos, corr)\}$ if $isAnchor$ else $\{\}$ **update**
2: **for each** neighbour i **and** anchor a in dpc_i **do**
3: increase $dist_i(a)$ by $nbrDist_i$ in $dpc_i(a) = (dist_i(a), pos_i(a), corr_i(a))$
4: **end for**
5: $dpc \leftarrow$ merge of dpc_i for each neighbour i, preferring shorter distances
6: **if** size of dpc is larger than K **then**
7: $dpc \leftarrow$ select K elements of dpc with smallest distance
8: **end if**
9: **return** dpc
10: **end share**
11: **return** $dist_a, pos_a, corr_a = dpc(a)$ for each anchor a in dpc

```
def anchorsDPC(pos, corr, isAnchor, nbrDist, K) {
  val init = if (isAnchor) {map(uid(), tuple(0,pos,corr))} else {map()};
  share(init, (ndpc) => {
    val mdpc = nmap(map_inc, ndpc, nbrDist);
    val dpc = nfold(map_join, mdpc, init);
    map_trunc(dpc, K)
  })
}
```

Fig. 4. K-closest anchors DPC algorithm (above) and the corresponding XC code (below). Functions `map_inc`, `map_join` and `map_trunc` respectively increase the first component of tuples in a map, merge maps preferring smaller values for the first component, and select the K values in the map with the smallest first component.

and, if the total number exceeds a maximum value K, it selects only the K closest anchors in terms of estimated distance. Regardless of the variant chosen, the DPC information collected from the anchors is then used to update *pos* and *corr*, as shown in the following lines of the algorithm. If the node is an anchor (lines 4–6), the position is known and no calculation is needed to update *pos*. Instead, *corr* is set to the ratio between the sum of Euclidean distances and the sum of estimated distances to the other anchors, providing the needed scaling factor from estimated to Euclidean. For non-anchor nodes, there is no associated correction factor, and *corr* is hence just set to 1. Instead, *pos* is updated through multilateration (lines 9), where the distances to anchors are obtained by multiplying estimated distances and correction factors (line 8).

In the R-Close Anchors variant (Figure 3), a **spawn** of a new aggregate process is performed for each anchor (line 1) to handle data collection concurrently. Each process checks the estimated distance to the anchor using distance estimates to neighbours *nbrDist* (line 2) and performs a broadcast of the anchor's position and correction factor (line 3). If the distance exceeds the predefined threshold R, the current node exits the aggregate process (line 4), ensuring that only "close enough" anchors are considered. This is crucial to ensure self-

stabilisation: if a node stops being an anchor, distance estimates from it will start increasing until when its process is fully terminated. The algorithm terminates (line 7) returning a collection of distance, position and correction values for each anchor whose process was active in the current device (thus, only those within a distance of R).

In the K-Closest Anchors variant (Figure 4), the node maintains a distributed table of anchor information using the **share** construct (line 1). For each neighbour i and anchor a in the table received from that neighbour (line 2), the node updates the distance estimates in that table by adding the distance estimate to the neighbour *nbrDist* (line 3). All tables received are then merged, preferring shorter distances if the same anchor appears in multiple tables (line 5). If the total number of anchors in the resulting table *dpc* exceeds the maximum value K (lines 6–7), only the K closest anchors in it are retained. Finally, the selected anchors are shared with neighbours (line 9), and returned to be used in the subsequent multilateration step (line 11).

In both variants, distances between nodes and anchors are expressed according to a given metric *nbrDist*, which may take very different forms: for instance, it may only represent the number of hops (if *nbrDist* is equal to 1 for each neighbour) or on noisy estimated distances (for instance, calculated with power ratio RSSI or message round-trip time measurements). The correction factor is crucial to ensure that any such metric can be used to estimate distances, leveraging the exact positional knowledge of anchors to compensate for systematic errors or even totally different scales, such as when distances are measured in hops. Furthermore, both variants can be shown to be self-stabilising.

Theorem 2. *Algorithm 2 is self-stabilising with either sub-routines in Algorithm 3 or Algorithm 4.*

Proof. Algorithm 3 is self-stabilising since it is a `spawn` on a fixed (thus stable) set of keys, whose process body consists of the *distance* and *broadcast* algorithms together with local-only computations, which are all known from literature to be self-stabilising [15].

Algorithm 4 is also self-stabilising, since it is an instance of the minimising pattern presented in [15]. In this pattern, a `share` repeatedly updates partially ordered values by first *increasing* with an increasing function all values received by neighbours, and then returning their *join* (maximum lower bound). This is exactly the structure of Algorithm4, where:

- values are maps of at most K elements or the form $UID \mapsto (dist, pos, corr)$;
- values are partially ordered by $m \leq m'$ iff for each $\delta \in \mathbf{dom}(m) \cap \mathbf{dom}(m')$ we have that $m(\delta)_1 \leq m'(\delta)_1$ and $m(\delta)_1 = m'(\delta)_1 \Rightarrow m(\delta) = m'(\delta)$ (for devices appearing in both maps, the distances in m are not larger than those in m', and if they are the same so are the position and correction values), and for each $\delta' \in \mathbf{dom}(m') \setminus \mathbf{dom}(m)$ we have that $m'(\delta')_1 \geq m(\delta)_1$ for all $\delta \in \mathbf{dom}(m)$ (the distances in m' for devices that do not appear in m are larger than all distances in m);

- notice that this partial order induces a *join* operation (maximum lower bound) that is exactly the K elements with smallest distance among both maps after removing duplicates by favoring shorter distances (which is what Algorithm 4 calculates in *dpc*);
- the increasing function is the one updating distances in a map m by adding them the values $nbrDist_i$, which is indeed increasing with respect to the partial order above.

Thus, Algorithm 2 obtains its *dpc* values from a self-stabilising algorithm, whose inputs are the positions *pos* of anchors (which is an algorithm input, so stable by hypothesis) and corrections of anchors *corr*, which are computed by the algorithm itself, so we have to prove that they are also self-stabilising. First, notice that both Algorithms 3 and 4 do not use the correction values to calculate distances and positions. It follows that even if corrections are not stable, those algorithms already self-stabilise for their computation of distances and positions in finite time. After distances and positions are stable, the correction factors of anchors as computed in Algorithm 2 also stabilise. At this point all inputs of Algorithms 3 and 4 are stable, so in additional finite time also the correction factors computed by those algorithms will be stable. Since the position computed by Algorithm 2 is a local function of those values, this concludes the self-stabilisation proof.

5 Evaluation

We evaluated the algorithms presented in this paper in the following variations. For *distance vector-based positioning*, we considered:

- multilateration with R-close anchors using aggregate processes, where R is sufficiently large to include all anchors in the area (in the following, we will call it `dv all`);
- multilateration with the six closest anchors (`dv 6close`).

For each of them we considered both a `hop` version (where distance measures to neighbours are not available, and hop-counts are used instead), and the `real` version (using noisy distance measures with neighbours). For *non-bayesian cooperative localisation*, we considered both the version from literature based on gradient descent (`nbcoop`) and our variation based on multilateration (`mlcoop`). These algorithms assume the availability of distance measures, so do not have a `hop` version. The algorithms and simulation setup were implemented in the FCPP library [4].

We considered three simulation parameters: *radius* (r), *speed* (v) and *variance* (d). The *radius* parameter, from 50m to 300m, determined the distance at which communication between devices becomes impossible, and indirectly affected the overall size of the network in hops (larger radius, fewer hops) and the average number of anchors in neighbourhoods (larger neighbourhoods contain more anchors). The *speed* parameter, from $0m/s$ to $5m/s$, determined the

speed of long-range movement for the devices. The *variance* parameter, from 0% to 50%, determined the relative standard error of the three sources of noise that we considered: *(i)* error in round timing; *(ii)* error in distance measures; *(iii)* variance in communication success rate (more details below). We considered the speed parameter only in isolation, using zero speed when varying the other parameters, since that allowed for trends to be observed more clearly in the plots.

We simulated a rectangular area of $500m \times 500m$, with 20 stationary anchors uniformly positioned along the border, and 100 devices positioned inside the area uniformly at random, which moved across randomly-generated checkpoints at a constant speed v. Devices performed rounds every second on average, with standard error d. Distance measures where computed by multiplying exact distances with a Weibull distribution of mean 1 and standard deviation d. We used a probabilistic model for connectivity based on real-world data, tuned to have an about 100% delivery success at 0m distance, about 0% delivery success beyond *radius* distance, and 50% delivery success at distance $r \cdot (1 - d)$. This entails that with $d = 0$, messages always succeed within r and fail above it; and as d increases more devices get an higher chance of communication failure.

In order to test the resiliency of the proposed algorithms, we simulated that 1/4 of the devices and anchors crash after $50s$ of simulated time, and are replaced (or fixed) after $100s$ of simulated time, until simulation ends at $150s$. We run simulations for 26 values of *radius*, 21 values of *speed*, 26 values of *variance*, and 100 different random seeds for each scenario, for a total of 7100 simulations. The experiments were run on a MacBook Pro 2019 with a 2.4GHz i9 8-core CPU and 32GB 2667MHz DDR4 RAM and required about 8 hours to complete. The experiment source code is publicly available online on Zenodo,[3] and can run both graphical simulations (see Figure 5) and plots from batch simulations (see Figure 6).

First, we inspected the size of messages exchanged for the various algorithms. The results had an average for `dv all` at 1889 bytes, `dv 6close` at 321 bytes, and both `coop` algorithms at 32 bytes, with almost zero variance across time and different scenarios so we do not include a plot for them (as it would just consist of horizontal lines). By profiling, we also inspected the time spent on the different algorithms during a simulation, obtaining about $4.6s$ for `dv all`, $1.1s$ for `dv 6close` and $0.3s$ for `coop` algorithms. Hop versions were a bit more efficient than their real counterparts, and `nbcoop` was a bit more efficient than `mlcoop`, but with a difference below 10%.

Then, we inspected the mean *error* (distance in meters between the estimated position and true position) among all devices, both as time passes in a default scenario (with $r = 150m, v = 0m/s, d = 20\%$) and as one of the parameters r, v or d vary (with other parameters at the default), averaged across all simulated time (see the four plots in Figure 6). The times when devices and anchors crash and then get replaced induce spikes in error that are then recovered (top-left graph). The `dv all` algorithms have significantly worst spikes, that are

[3] https://doi.org/10.5281/zenodo.19443874

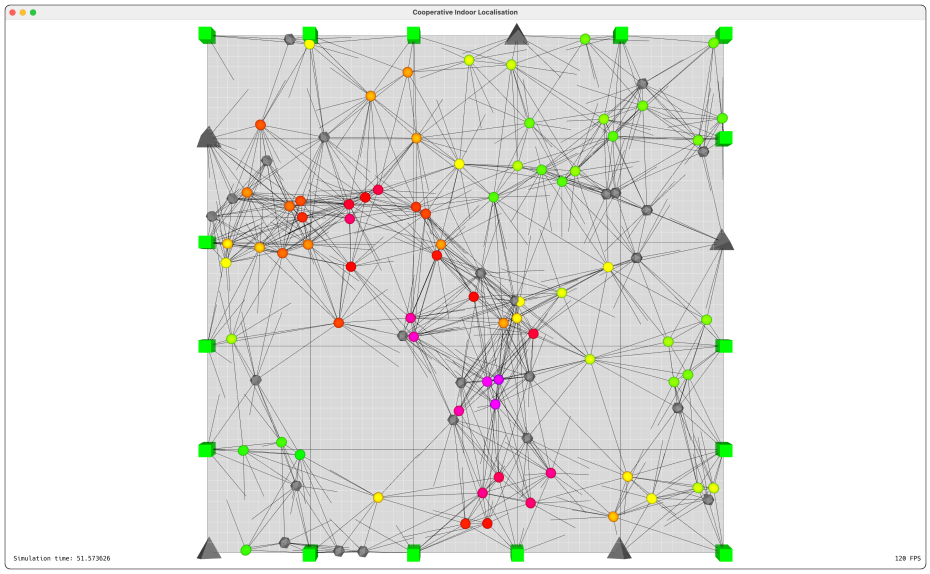

Fig. 5. Screenshot of the simulation with parameters $r = 150m$, $v = 0m/s$ and $d = 20\%$ at about $51s$ after the start. Crashed anchors and nodes are coloured in gray, with tetrahedron and icosahedron shapes respectively. The other anchors and nodes, with cubic and spherical shape respectively, are coloured according to the current error of the position estimated by the `nbcoop real` algorithm (hues changing continuously from green to yellow, red, purple with increasing error). Lines between nodes represent communication links (half lines represent asymmetric communication).

much lower for the other algorithms. Furthermore, both cooperative algorithms to not produce a spike in error when anchors are removed, while `mlcoop` and `dv 6close real` do not produce a spike when anchors are reintroduced. Generally, increasing *variance* (top-right plot) degrades performance as expected, but not by the same amount for each algorithm: `dv 6close hop` is the least affected as it is almost constant, probably partially because it doesn't rely on the increasingly noisier distance estimates. Reducing *radius* (bottom-left plot) also tends to degrade performance, as the network becomes increasingly disconnected: with smaller values, more and more devices resort to totally random estimates of their position, as they are not connected to any anchor. More unexpectedly, also a larger *radius* tends to degrade performance: this is probably due to the fact that an increased *radius* also reduces the hop-diameter of the network to a few units, so that `hop` algorithms have an hard time getting reasonable estimates. However, this also affects `real` algorithms (except for `mlcoop`), although by a smaller extent, for unclear reasons. Finally, increasing *speed* (bottom-right plot) also tends to degrade performance, although not significantly nor uniformily, proving the resilience in mutable environments of all algorithms we tested.

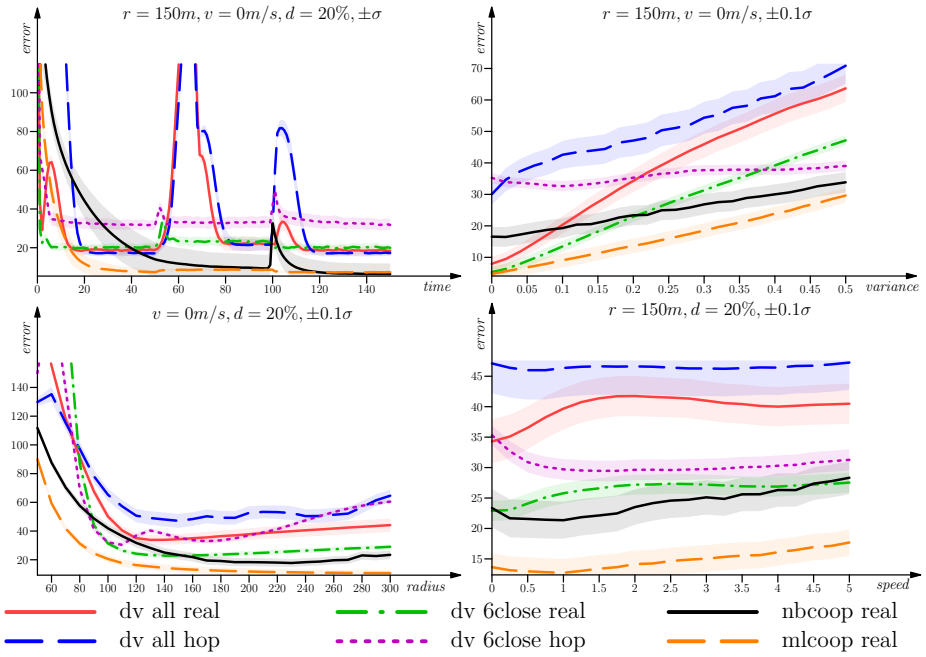

Fig. 6. Average localisation error over time (top-left, in seconds), variance (top-right, a dimensionless value from 0% to 50%), radius (bottom-left, in meters), and speed (bottom-right, in m/s). Lightly coloured bands represent the standard deviation of the results for the time plot, or 0.1 the standard deviation for the other plots (which have an higher deviation, since the average is calculated also over varying time points).

Summarizing the information across all plots, the `real` versions consistently outperform their `hops` counterparts, except for `dv 6close` when *variance* is very high (above 38%). Thus, using distance estimates proves to be useful whenever they are available, except when they are so noisy that they become effectively more inaccurate than just estimating hops.

Furthermore, `dv 6close` outperforms `dv all` almost always: the only exception is when *radius* is below $80m$ (so that the network is almost disconnected), or for the `hop` variants when *variance* is below 2%. Thus, considering only the closest anchors tends to improve performance, which can be justified since the distances to the farthest anchors are the least accurate. Given that `dv all` also requires much larger message sizes, its use is basically never advised.

Overall, the `mlcoop` algorithm is a clear winner in basically every scenario. This algorithm not only achieves the best steady-state values (together with `nbcoop`), but also provides the faster convergence time on simulation startup, while being almost unaffected by both the removal and addition of new devices. Also `nbcoop` performs quite well and is unaffected by device removal, but gets penalised by its slow initial convergence and by the error spike when new devices

are added. Although it is still the second best algorithm in most conditions, it is outperformed by `dv 6close real` for small *variance* and for *radius* between about $90m$ and $150m$. On the other hand, we remark that both `coop` algorithms are stabilising but not necessarily self-stabilising, since they may converge to a local minimum [8]. In summary, the best algorithms depending on the setting are:

- if distance estimates are not available, or formal self-stabilisation is required and variance is very high, `dv 6close hop`; otherwise
- if formal self-stabilisation is required, `dv 6close real`; otherwise
- if initial convergence is not an issue (long-running system), and computational power is very constrained, `nbcoop` provides the best steady-state values with small messages and few computations per round; otherwise
- `mlcoop` provides the best accuracy and resilience overall with small messages.

6 Conclusion

In this work, we addressed the problem of distributed multi-hop localisation in indoor environments, in which devices must estimate their position through local interactions, in the presence of measurement noise and dynamic network conditions. We reformulated two classical geometric approaches (DV-based positioning and NB-cooperative) within the aggregate computing paradigm, exploiting the exchange and alignment constructs of the XC language to obtain modular, composable, fully distributed and (self-)stabilising implementations. In addition to the versions more directly translated from the literature, we introduced and analysed improved variants: the K-Closest anchor selection strategies for DV-based positioning, and a variant of NB-cooperative based on multilateration instead of gradient descent. Through an extensive experimental set-up conducted within the FCPP framework, which considered different configurations of communication radius, mobility, and noise levels, we evaluated accuracy, convergence time, resilience to node crashes, message size, and computational cost. The results show that selecting only the closest anchors significantly improves the performance of DV-based variants. Furthermore, the multilateration-based cooperative version represents, overall, the solution with the best trade-off between accuracy, convergence speed, and robustness, while maintaining small message sizes and limited computational costs.

Despite the encouraging results, this work presents some limitations that open interesting directions for future research. First, the analysis focused on two representative classical algorithms and related variants, leaving room for the integration of additional cooperative localisation strategies. Among these, a weighted version of the *mlcoop* algorithm appears particularly promising. In such version, each node also maintains an estimate of its own error and uses it as a weight in the multilateration process. This may modulate the contribution of different nodes according to their reliability and potentially improve robustness in scenarios with high variability or highly noisy measurements. Furthermore, the guaranteed self-stabilising version of the *coop* algorithm inspired from [8],

albeit with expected slower convergence, should also be evaluated. Second, the evaluations were conducted in a highly synthetic simulated environment, although extensive and parametric. A natural evolution of this work could consist in defining a more realistic case study, modelled on a concrete application scenario and aimed at a future experimental validation in the field. This would make it possible to analyse more deeply aspects such as hardware constraints, energy consumption, radio interference, and real environmental dynamics, further consolidating the practical applicability of the proposed solutions.

Acknowledgments. This publication is part of the project NODES, which has received funding from the MUR – M4C2 1.5 of PNRR funded by the European Union - NextGenerationEU (Grant agreement no. ECS00000036). This study has been supported by the Italian PRIN project "CommonWears" (2020HCWWLP) and was carried out within the Agritech National Research Center and received funding from the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR) – MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.4 – D.D. 1032 17/06/2022, CN00000022). This manuscript reflects only the authors' views and opinions, neither the European Union nor the European Commission can be considered responsible for them.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Asmaa, L., Aroussi, H.K., Mouloudi, A.: Localization algorithms research in wireless sensor network based on multilateration and trilateration techniques. In: Mohajir, M.E., Achhab, M.A., Chahhou, M. (eds.) Third IEEE International Colloquium in Information Science and Technology, CIST 2014, Tetouan, Morocco, October 20-22, 2014. pp. 415–419. IEEE (2014). https://doi.org/10.1109/CIST.2014.7016656, https://doi.org/10.1109/CIST.2014.7016656
2. Audrito, G., Casadei, R., Damiani, F., Salvaneschi, G., Viroli, M.: The exchange calculus (XC): A functional programming language design for distributed collective systems. J. Syst. Softw. **210**, 111976 (2024). https://doi.org/10.1016/J.JSS.2024.111976
3. Audrito, G., Casadei, R., Damiani, F., Torta, G., Viroli, M.: Programming distributed collective processes in the exchange calculus. Log. Methods Comput. Sci. **21**(4) (2025). https://doi.org/10.46298/LMCS-21(4:3)2025
4. Audrito, G., Torta, G.: FCPP to aggregate them all. Sci. Comput. Program. **231**, 103026 (2024). https://doi.org/10.1016/J.SCICO.2023.103026, https://doi.org/10.1016/j.scico.2023.103026
5. Basterrech, S., Mohamed, S., Rubino, G., Soliman, M.A.: Levenberg - marquardt training algorithms for random neural networks. Comput. J. **54**(1), 125–135 (2011). https://doi.org/10.1093/COMJNL/BXP101, https://doi.org/10.1093/comjnl/bxp101
6. Beal, J., Pianini, D., Viroli, M.: Aggregate programming for the Internet of Things. IEEE Computer **48**(9) (2015). https://doi.org/10.1109/MC.2015.261

7. Casadei, R., Viroli, M., Audrito, G., Pianini, D., Damiani, F.: Engineering collective intelligence at the edge with aggregate processes. Eng. Appl. Artif. Intell. **97**, 104081 (2021). https://doi.org/10.1016/j.engappai.2020.104081
8. Morano, G., Hrovat, A., Švigelj, A., Velkaverh, J., Žnidaršič, P., Javornik, T.: Experimental performance evaluation of cooperative localization in indoor environment. IEEE Sensors Journal **25**(23), 43426–43445 (2025). https://doi.org/10.1109/JSEN.2025.3620118
9. Niculescu, D., Nath, B.: DV based positioning in ad hoc networks. Telecommun. Syst. **22**(1-4), 267–280 (2003). https://doi.org/10.1023/A:1023403323460, https://doi.org/10.1023/A:1023403323460
10. Qiao, J., Yang, F., Liu, J., Huang, G., Zhang, W., Li, M.: Advancements in indoor precision positioning: A comprehensive survey of uwb and wi-fi rtt positioning technologies. Network **4**(4), 545–566 (2024). https://doi.org/10.3390/network4040027, https://www.mdpi.com/2673-8732/4/4/27
11. Shams, R., Otero, P., Aamir, M., Khan, F.H.: Joint algorithm for multi-hop localization and time synchronization in underwater sensors networks using single anchor. IEEE Access **9**, 27945–27958 (2021). https://doi.org/10.1109/ACCESS.2021.3058160, https://doi.org/10.1109/ACCESS.2021.3058160
12. Simonetto, A., Leus, G.: Distributed maximum likelihood sensor network localization. IEEE Transactions on Signal Processing **62**(6), 1424–1437 (2014). https://doi.org/10.1109/TSP.2014.2302746
13. Testa, L., Audrito, G., Damiani, F., Torta, G.: Aggregate processes as distributed adaptive services for the industrial internet of things. Pervasive Mob. Comput. **85**, 101658 (2022). https://doi.org/10.1016/j.pmcj.2022.101658
14. Thomas, F., Ros, L.: Revisiting trilateration for robot localization. IEEE Trans. Robotics **21**(1), 93–101 (2005). https://doi.org/10.1109/TRO.2004.833793, https://doi.org/10.1109/TRO.2004.833793
15. Viroli, M., Audrito, G., Beal, J., Damiani, F., Pianini, D.: Engineering resilient collective adaptive systems by self-stabilisation. ACM Trans. Model. Comput. Simul. **28**(2), 16:1–16:28 (2018). https://doi.org/10.1145/3177774
16. Viroli, M., Beal, J., Damiani, F., Audrito, G., Casadei, R., Pianini, D.: From distributed coordination to field calculus and aggregate computing. J. Log. Algebraic Methods Program. **109** (2019). https://doi.org/10.1016/j.jlamp.2019.100486
17. Wymeersch, H., Lien, J., Win, M.Z.: Cooperative localization in wireless networks. Proc. IEEE **97**(2), 427–450 (2009). https://doi.org/10.1109/JPROC.2008.2008853, https://doi.org/10.1109/JPROC.2008.2008853
18. Xiong, Y., Wu, N., Shen, Y., Win, M.Z.: Cooperative localization in massive networks. IEEE Transactions on Information Theory **68**(2), 1237–1258 (2022). https://doi.org/10.1109/TIT.2021.3126346